SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

Faculty of Science and Humanities

Tiruchirappalli, Tamil Nadu-621 105

Department of Computer Applications

PRACTICAL RECORD

NAME :

REGISTER NO : RA25322410500

COURSE : MCA(GENERAL)

SEMESTER/ YEAR : II / I

SUBJECT CODE : PCA25S02J

SUBJECT NAME : INTERNET OF THINGS (IOT)

April 2026

SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

Faculty of Science and Humanities

Tiruchirappalli, Tamil Nadu-621 105

Department of Computer Applications

BONAFIDE CERTIFICATE

This is to certify that the Bonafide work is done by Mr/Ms. _____ Register No.RA25322410500____in **Internet of Things (IoT) (Subject Code: PCA25S02J)** at Computer Lab, SRMIST, Tiruchirappalli, in April 2026.

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for the University Practical Examination held at SRMIST, Tiruchirappalli, Department of Computer Applications on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INDEX

S. No.	Date	Name of the Program	Page No.	Signature
1		Arduino Installation and Blink LED using NodeMCU ESP8266	4	
2		Interfacing IR (Infrared Proximity) Sensor with NodeMCU ESP8266	6	
3		Interfacing Light Sensor (LDR) with NodeMCU ESP8266	8	
4		Interfacing Temperature & Humidity Sensor (DHT11) with NodeMCU ESP8266	10	
5		Control an LED using Cloud BLYNK APP	12	
6		Interfacing Soil Moisture Sensor with NodeMCU ESP8266	14	
7		Interfacing MQ-4 Gas Sensor with NodeMCU ESP8266	16	
8		Interfacing Relay and Control LIGHT with NodeMCU ESP8266	18	
9		Working of Raspberry Pi	20	
10		Create a Webserver using NodeMCU ESP8266	22	
11		Home Automation – Level 0	24	
12		Home Automation – Level 4	26	
13		Smart Irrigation System	28	
14		Weather Reporting System	30	

Practice 1

Arduino Installation and Blink LED using NodeMCU ESP8266

Aim: To install the Arduino IDE, configure it for NodeMCU ESP8266, and program the board to blink an LED.

Duration: 2 Hours

Theory

The NodeMCU ESP8266 is a low-cost Wi-Fi microcontroller based on the ESP8266 chip by Espressif Systems. It can be programmed via the Arduino IDE after installing the ESP8266 board package. The onboard LED is connected to GPIO2 (D4) and is active-LOW — it turns ON when the pin is driven LOW. The two mandatory Arduino functions are setup() (runs once) and loop() (runs repeatedly).

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266 (V3)	1
2	USB Type-A to Micro-USB Cable	1
3	LED (Red/Green)	1
4	Resistor 220Ω	1
5	Breadboard	1
6	Jumper Wires	5
7	PC with Arduino IDE 2.x	1

Circuit Diagram

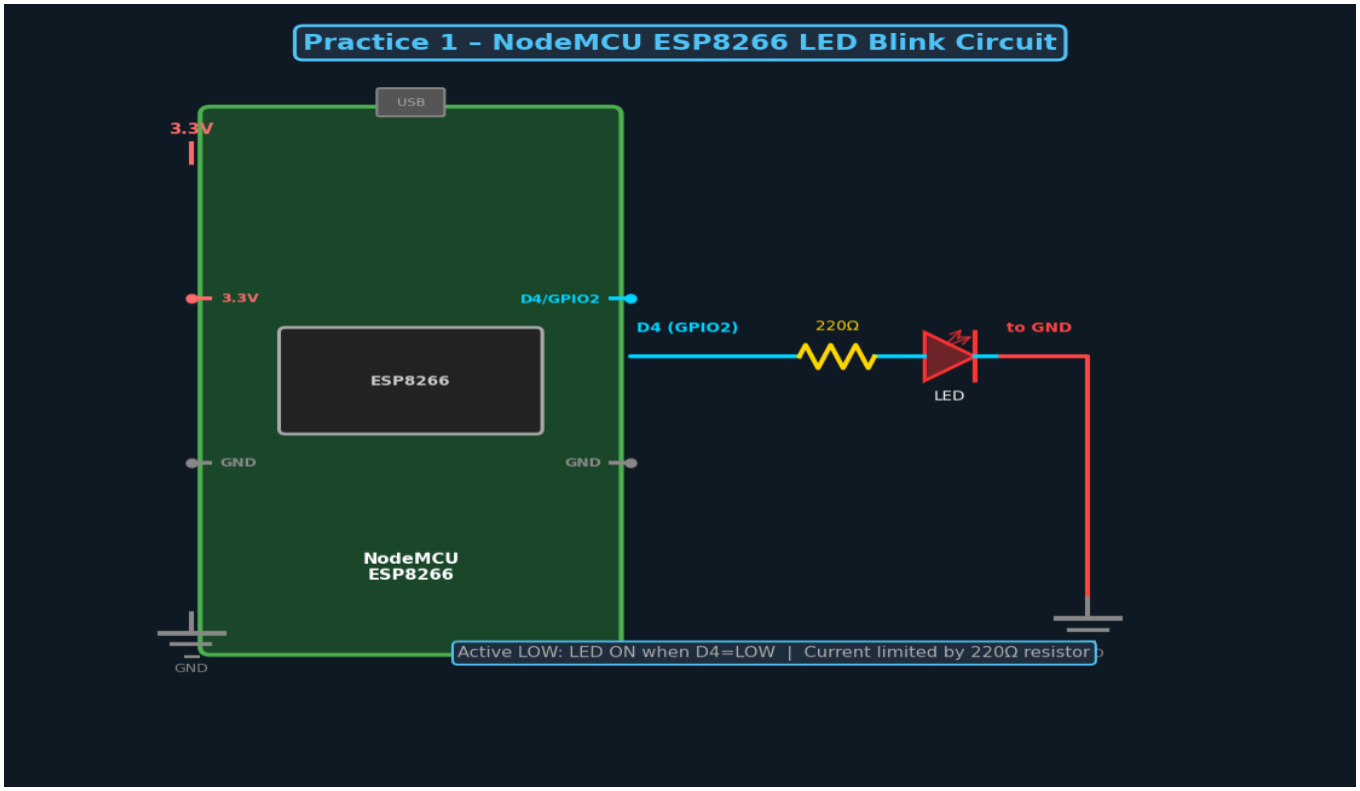


Fig 1.1 – NodeMCU ESP8266 LED Blink Circuit

Arduino IDE Setup

1. Download Arduino IDE 2.x from https://www.arduino.cc/en/software
2. File → Preferences → Additional Board Manager URLs: http://arduino.esp8266.com/stable/package_esp8266com_index.json
3. Tools → Board → Boards Manager → search 'ESP8266' → Install
4. Tools → Board → NodeMCU 1.0 (ESP-12E Module)
5. Tools → Port → select COM port of NodeMCU

Program Code

```
// Practice 1: Blink LED on NodeMCU ESP8266
#define LED_PIN D4   // GPIO2 - onboard LED (active LOW)

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, LOW);   // LED ON
  Serial.println("LED ON");
  delay(1000);
  digitalWrite(LED_PIN, HIGH);  // LED OFF
  Serial.println("LED OFF");
  delay(1000);
}
```

Expected Output

- LED blinks at 1-second ON / 1-second OFF intervals.
- Serial Monitor (115200 baud) shows 'LED ON' and 'LED OFF' alternately.

Viva Questions

- What is the operating voltage of NodeMCU ESP8266?
- Why is a 220Ω resistor used with an LED?
- What does active-LOW mean for the onboard LED?

Practice 2

Interfacing IR (Infrared Proximity) Sensor with NodeMCU ESP8266

Aim: To interface an IR proximity sensor and detect the presence of an object.

Duration: 2 Hours

Theory

An IR proximity sensor transmits infrared light and detects the reflected light from a nearby object. The DO (Digital Output) pin goes LOW when an object is detected (active LOW). The sensor's sensitivity range (2–30 cm) is adjusted via an onboard potentiometer. These sensors are widely used in obstacle detection, counting, and automation.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	IR Proximity Sensor Module (FC-51)	1
3	LED	1
4	Resistor 220Ω	1
5	Breadboard & Jumper Wires	1 set

Circuit Diagram

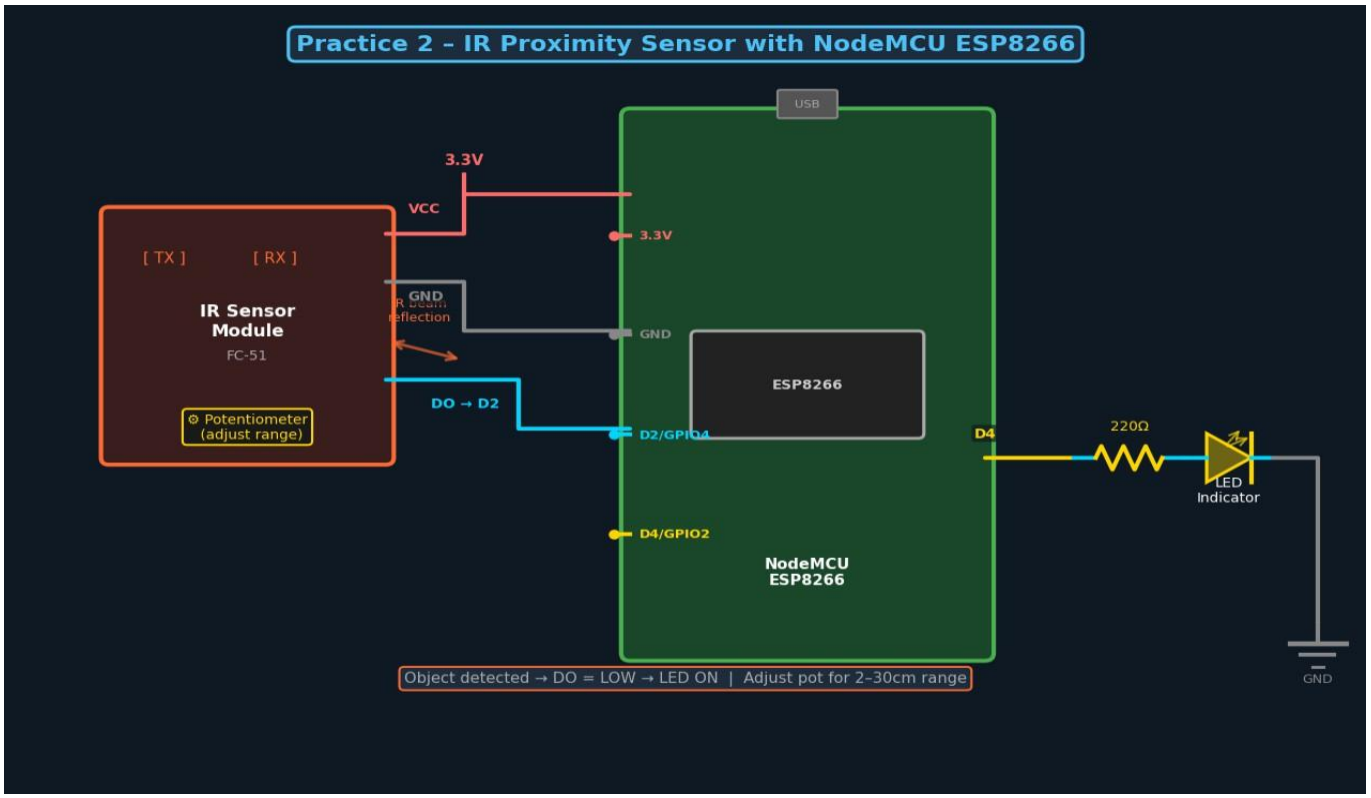


Fig 2.1 – IR Proximity Sensor Interface Circuit

Program Code

```
// Practice 2: IR Proximity Sensor
#define IR_PIN  D2
```

```
#define LED_PIN D4

void setup() {
  Serial.begin(115200);
  pinMode(IR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  if (digitalRead(IR_PIN) == LOW) {  // Object detected
    digitalWrite(LED_PIN, LOW);
    Serial.println("Object Detected!");
  } else {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("No Object");
  }
  delay(500);
}
```

Expected Output

- Object placed in front: 'Object Detected!' – LED turns ON.
- No object: 'No Object' – LED stays OFF.

Practice 3

Interfacing Light Sensor (LDR) with NodeMCU ESP8266

Aim: To interface an LDR and measure ambient light intensity using NodeMCU's analog input.

Duration: 2 Hours

Theory

An LDR (Light Dependent Resistor) decreases in resistance as light intensity increases. A voltage divider circuit with a fixed 10kΩ resistor converts this resistance change into a voltage. NodeMCU's A0 pin reads 0–1023 (10-bit ADC, 0–1V range). The mapped percentage represents light intensity from 0% (dark) to 100% (bright).

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	LDR Module (or LDR + 10kΩ resistor)	1
3	Breadboard & Jumper Wires	1 set

Circuit Diagram

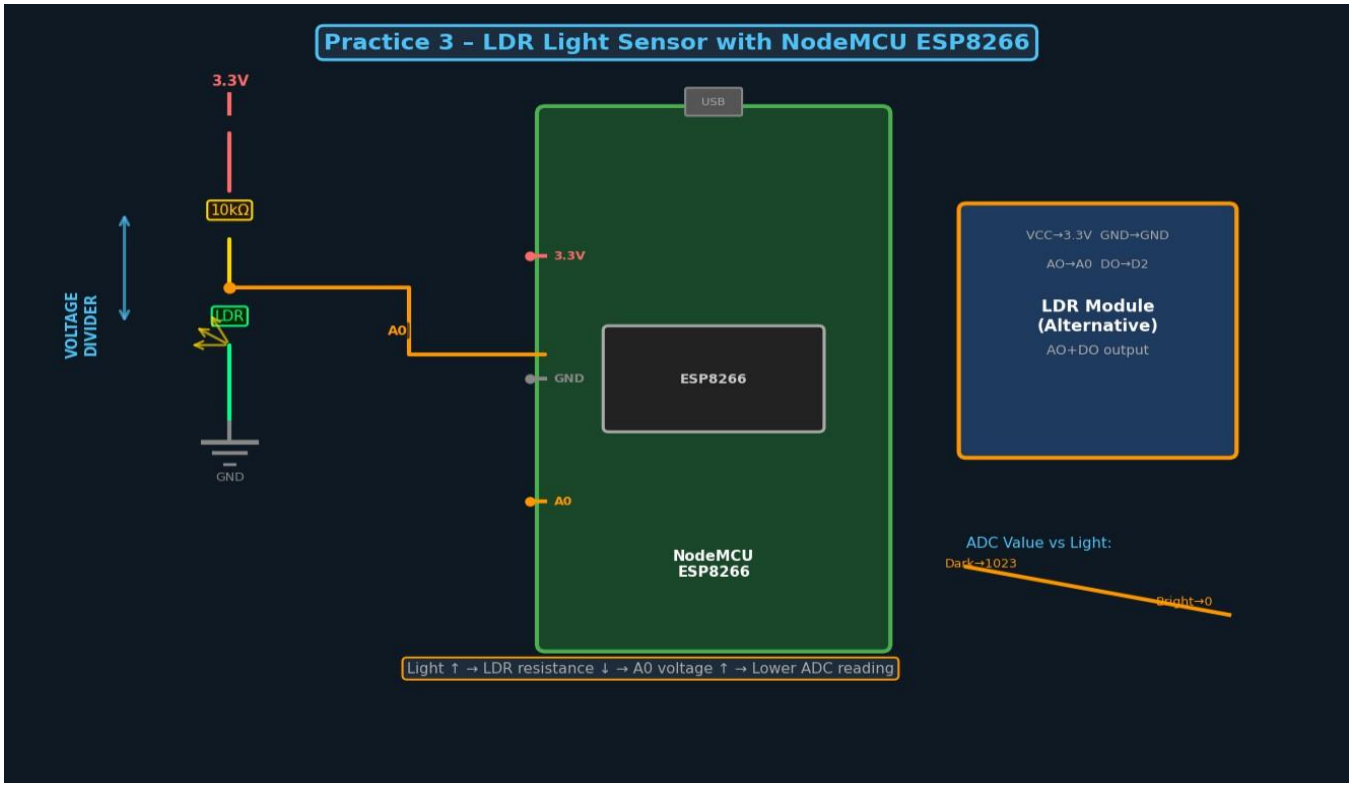


Fig 3.1 – LDR Voltage Divider Circuit with NodeMCU

Program Code

```
// Practice 3: LDR Light Sensor
#define LDR_PIN A0

void setup() {
  Serial.begin(115200);
}
```

```
void loop() {
  int raw = analogRead(LDR_PIN);
  int pct = map(raw, 0, 1023, 0, 100);
  Serial.print("Raw: "); Serial.print(raw);
  Serial.print(" | Light: "); Serial.print(pct); Serial.println("%");
  if (pct < 30) Serial.println("Dark – lights needed!");
  delay(1000);
}
```

Practice 4

Interfacing Temperature & Humidity Sensor (DHT11)

Aim: To interface a DHT11 sensor and display temperature/humidity readings via Serial Monitor.
Duration: 2 Hours

Theory

The DHT11 measures temperature (0–50°C, ±2°C) and humidity (20–90% RH, ±5%). It communicates over a single-wire protocol, sending a 40-bit data frame. The Adafruit DHT library simplifies this. A 4.7kΩ pull-up resistor on the DATA pin ensures reliable communication.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	DHT11 Sensor Module	1
3	Resistor 4.7kΩ (bare sensor only)	1
4	Breadboard & Jumper Wires	1 set

Circuit Diagram

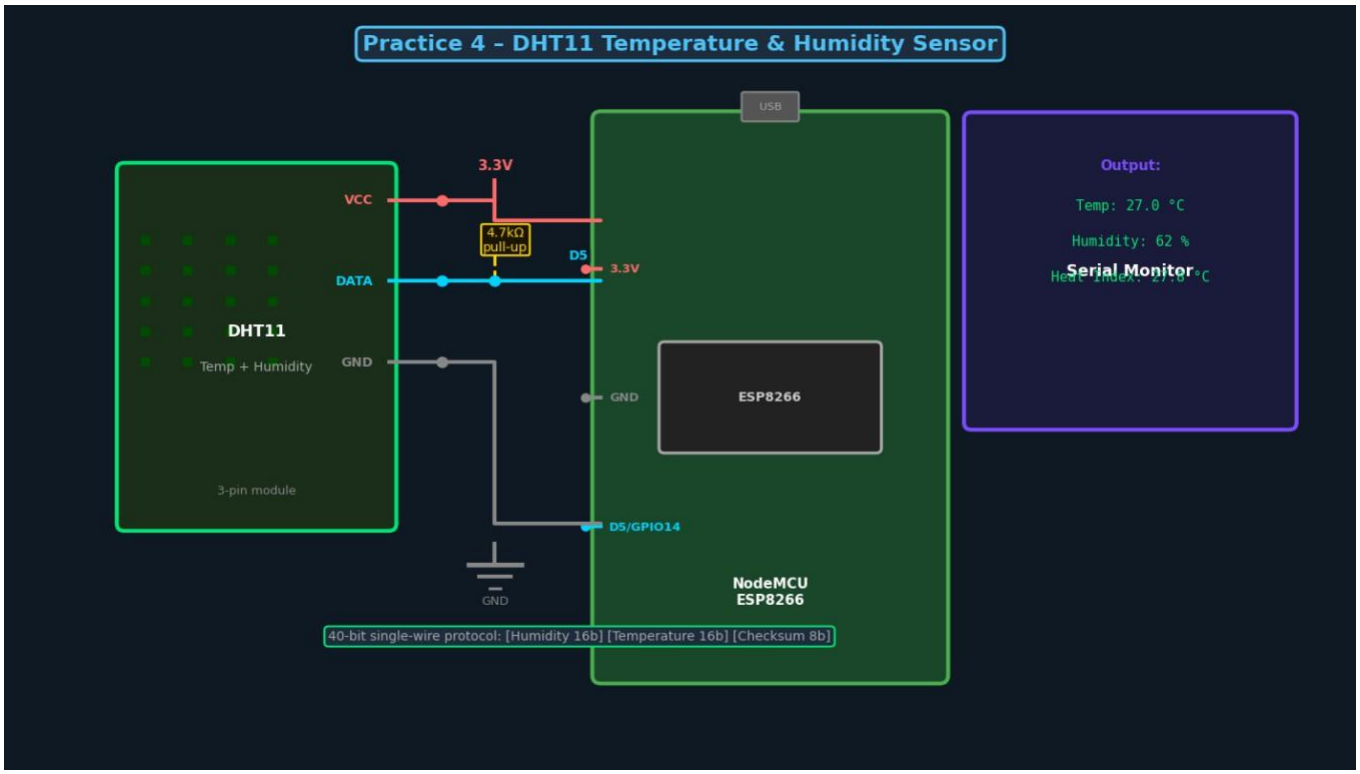


Fig 4.1 – DHT11 Sensor Connection with NodeMCU ESP8266

Library Installation

- Sketch → Include Library → Manage Libraries → search 'DHT sensor library' by Adafruit → Install

Program Code

```
// Practice 4: DHT11 Temperature & Humidity
```

```
#include <DHT.h>
#define DHTPIN  D5
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  delay(2000);
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) { Serial.println("Read error!"); return; }
  Serial.print("Humidity: ");    Serial.print(h);    Serial.print(" % ");
  Serial.print("Temperature: "); Serial.print(t);    Serial.println(" C");
}
```

Practice 5

Control an LED using Cloud BLYNK APP

Aim: To control an LED remotely from anywhere using the Blynk IoT cloud platform.

Duration: 3 Hours

Theory

Blynk is an IoT platform that connects hardware to mobile apps via the cloud. The NodeMCU authenticates with the Blynk server using an Auth Token and connects over Wi-Fi. A button widget in the app sends commands to a virtual pin, which the NodeMCU receives through the Blynk library's callback (BLYNK_WRITE). This enables control from anywhere with an internet connection.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	LED	1
3	Resistor 220Ω	1
4	Breadboard & Jumper Wires	1 set
5	Smartphone with Blynk App	1
6	Wi-Fi Internet Access	1

System Diagram

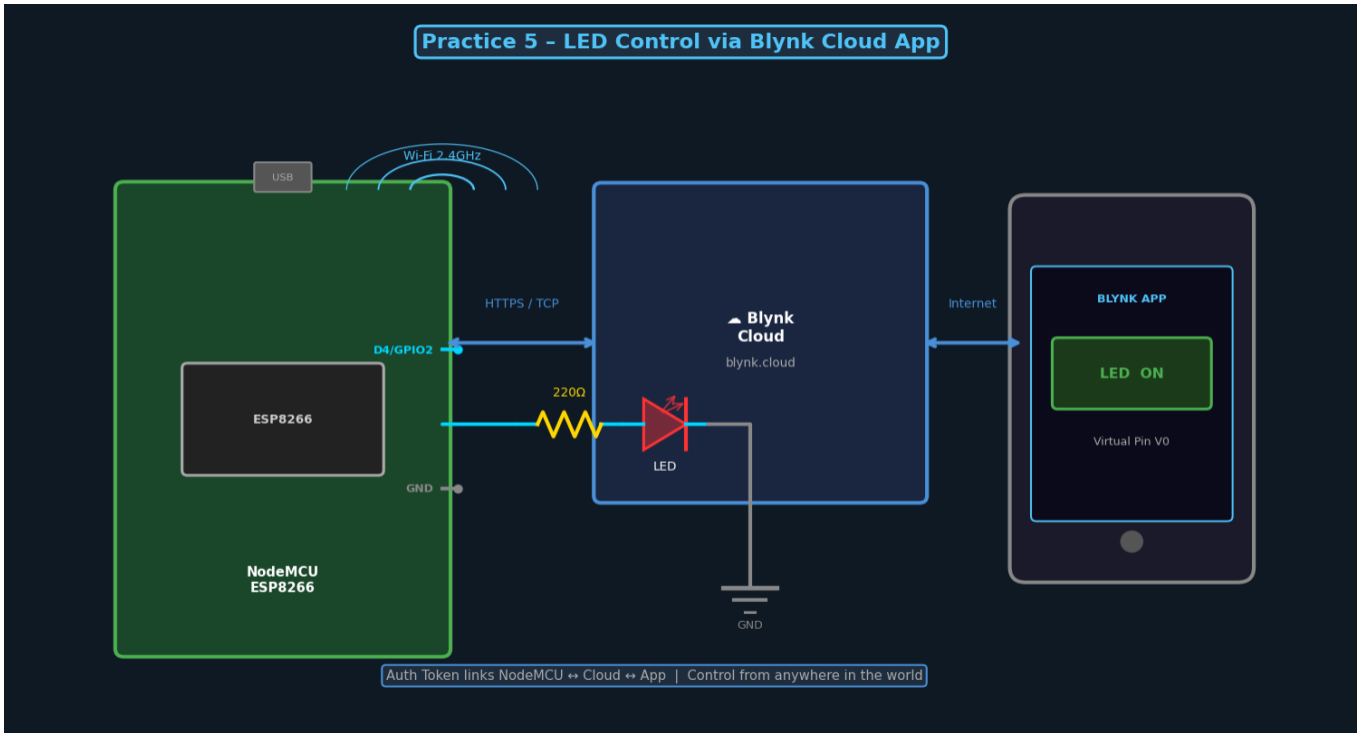


Fig 5.1 – Blynk Cloud LED Control Architecture

Blynk Setup Steps

6. Install 'Blynk IoT' from Google Play / App Store.

7. Create account at blynk.cloud → New Template → NodeMCU ESP8266.
8. Copy Auth Token, create Button widget mapped to Virtual Pin V0.
9. Install Blynk library in Arduino IDE → paste Auth Token in code.

Program Code

```
// Practice 5: Blynk LED Control
#define BLYNK_TEMPLATE_ID "YourTemplateID"
#define BLYNK_AUTH_TOKEN   "YourAuthToken"
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
char ssid[] = "WiFi_SSID";
char pass[] = "WiFi_Pass";
#define LED_PIN D4

BLYNK_WRITE(V0) {
  digitalWrite(LED_PIN, param.asInt());
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}
void loop() { Blynk.run(); }
```

Practice 6

Interfacing Soil Moisture Sensor with NodeMCU ESP8266

Aim: To monitor soil moisture level and trigger irrigation alerts automatically.

Duration: 2 Hours

Theory

Resistive soil moisture sensors measure electrical resistance between two probes. Wet soil has lower resistance (higher conductivity), producing a lower analog voltage. The AO pin gives analog output (0–1023) and DO gives a digital HIGH/LOW based on a threshold set by the onboard potentiometer. A capacitive sensor measures the dielectric constant of soil, which is more durable as it avoids electrolysis.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	Soil Moisture Sensor Module (YL-69 / FC-28)	1
3	Breadboard & Jumper Wires	1 set
4	Soil / wet paper towel for testing	1

Circuit Diagram

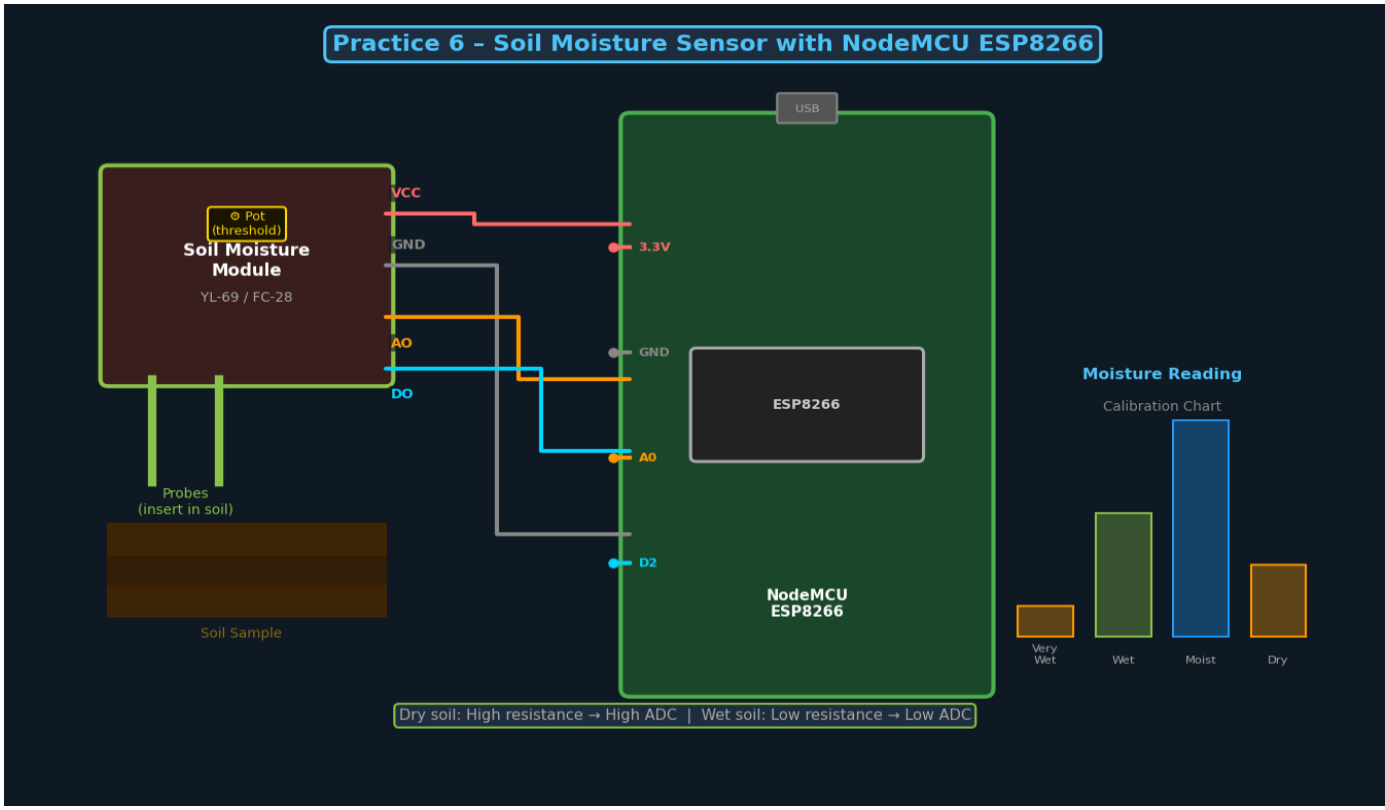


Fig 6.1 – Soil Moisture Sensor Connection with NodeMCU

Program Code

```
// Practice 6: Soil Moisture Sensor
#define SOIL_AO A0
```

```
#define SOIL_DO D2

void setup() {
  Serial.begin(115200);
  pinMode(SOIL_DO, INPUT);
}

void loop() {
  int raw = analogRead(SOIL_AO);
  int pct = map(raw, 1023, 0, 0, 100);
  Serial.print("Moisture: "); Serial.print(pct); Serial.println("%");
  if (pct < 30) Serial.println(">> ALERT: Irrigation needed!");
  delay(2000);
}
```

Practice 7

Interfacing MQ-4 Gas Sensor with NodeMCU ESP8266

Aim: To detect methane/natural gas leaks and trigger a buzzer alarm using MQ-4 sensor.

Duration: 2 Hours

Theory

The MQ-4 is a chemi-resistor gas sensor using tin oxide (SnO2) as the sensing material. Its resistance decreases in the presence of methane (CH4) and natural gas. It outputs an analog voltage proportional to gas concentration (AO pin) and a digital signal when concentration exceeds a threshold (DO pin). A 3-minute warm-up period is required before accurate readings.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	MQ-4 Gas Sensor Module	1
3	Active Buzzer	1
4	LED (Red) + 220Ω resistor	1
5	Breadboard & Jumper Wires	1 set

Circuit Diagram

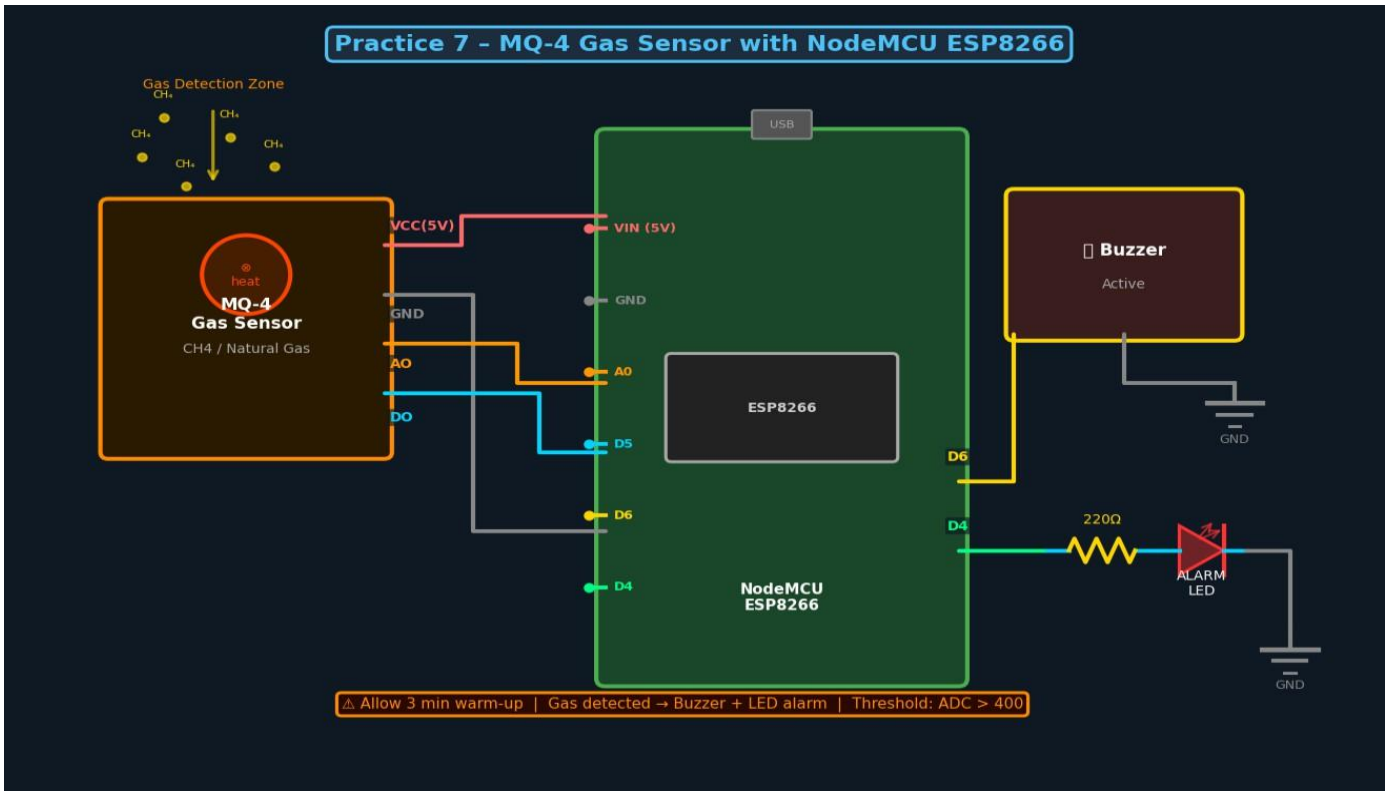


Fig 7.1 – MQ-4 Gas Sensor with Alarm Circuit

Program Code

```
// Practice 7: MQ-4 Gas Sensor
#define MQ4_AO  A0
```

```
#define  MQ4_DO  D5
#define  BUZZER  D6
#define LED_PIN D4

void setup() {
  Serial.begin(115200);
  pinMode(BUZZER, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("Warming up (3 min)...");
  delay(3000); // Use 180000 for production
}

void loop() {
  int gas = analogRead(MQ4_AO);
  Serial.print("Gas Level: "); Serial.println(gas);
  if (gas > 400 || digitalRead(MQ4_DO) == LOW) {
    Serial.println("*** GAS LEAK DETECTED! ***");
    digitalWrite(BUZZER, HIGH);
    digitalWrite(LED_PIN, LOW);
  } else {
    digitalWrite(BUZZER, LOW);
    digitalWrite(LED_PIN, HIGH);
  }
  delay(1000);
}
```

Practice 8

Interfacing Relay and Controlling LIGHT with NodeMCU ESP8266

Aim: To interface a 5V relay module with NodeMCU ESP8266 to switch an AC light bulb on and off.
Duration: 2 Hours

Theory

A relay is an electromechanical switch controlled by a small signal. The relay module uses a 5V coil triggered by NodeMCU's GPIO (via optocoupler for isolation), and can switch loads up to 10A at 250V AC. Most modules are active-LOW. The relay has three AC terminals: NO (Normally Open), NC (Normally Closed), and COM. Connect the load between NO and COM for normally-off operation.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	5V Single-Channel Relay Module	1
3	AC Light Bulb with Holder	1
4	AC Power Cord	1
5	Jumper Wires	4

Circuit Diagram

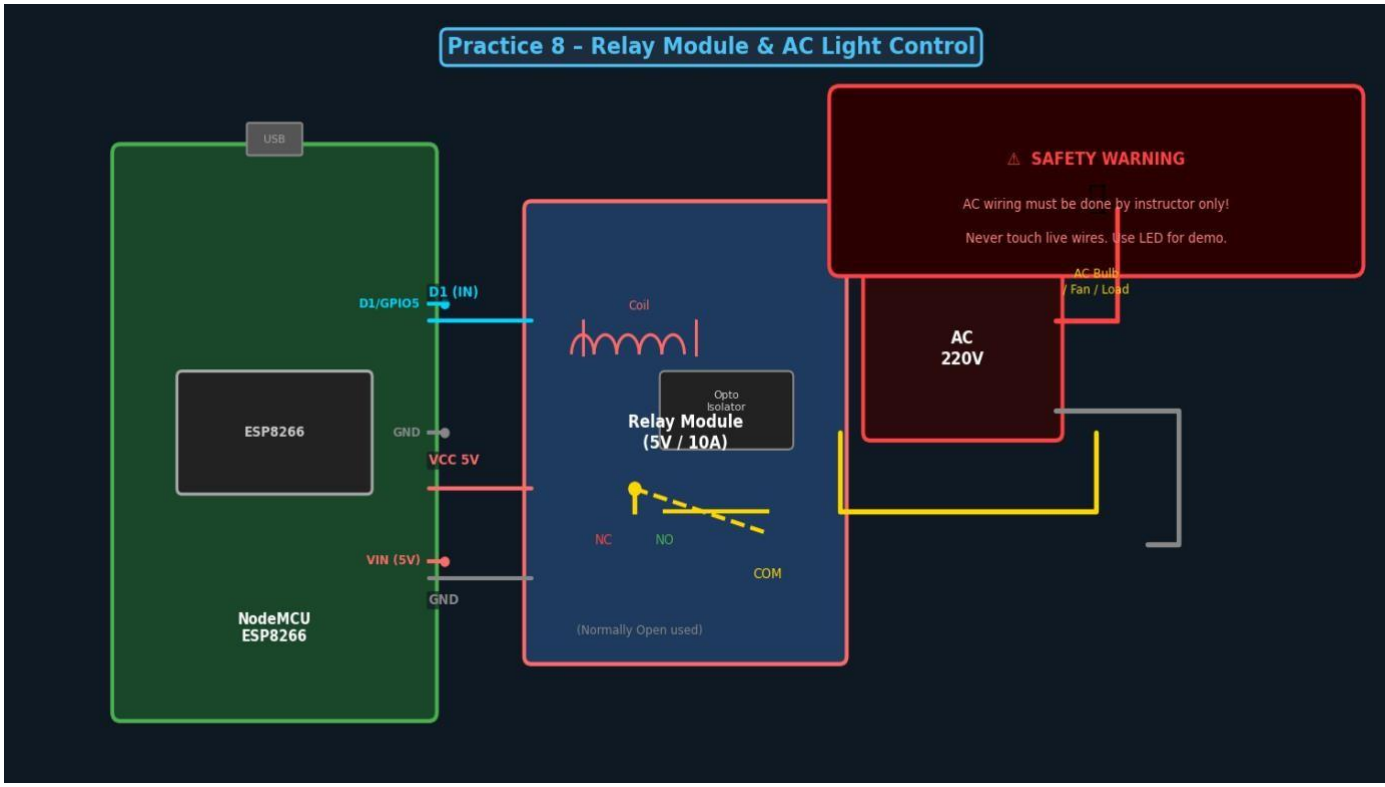


Fig 8.1 – Relay Module AC Light Control Circuit

⚠ SAFETY: AC wiring must be performed by the lab instructor only. Never touch live wires.

Program Code

```
// Practice 8: Relay Control for AC Light
#define RELAY_PIN D1  // Active LOW relay

void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, HIGH); // OFF initially
}

void loop() {
  Serial.println("Light ON");
  digitalWrite(RELAY_PIN, LOW);   // Relay energized
  delay(5000);
  Serial.println("Light OFF");
  digitalWrite(RELAY_PIN, HIGH);  // Relay de-energized
  delay(5000);
}
```

Practice 9

Working of Raspberry Pi

Aim: To understand Raspberry Pi architecture, set it up with Raspberry Pi OS, and control GPIO with Python.

Duration: 3 Hours

Theory

Raspberry Pi is a credit-card-sized single-board computer (SBC) running full Linux (Raspberry Pi OS). Unlike NodeMCU, it has a true ARM CPU, RAM, GPU, HDMI, USB, and a 40-pin GPIO header. It can run complex applications (web servers, AI, databases) while still interfacing with hardware sensors and actuators.

Raspberry Pi 4B Board Diagram

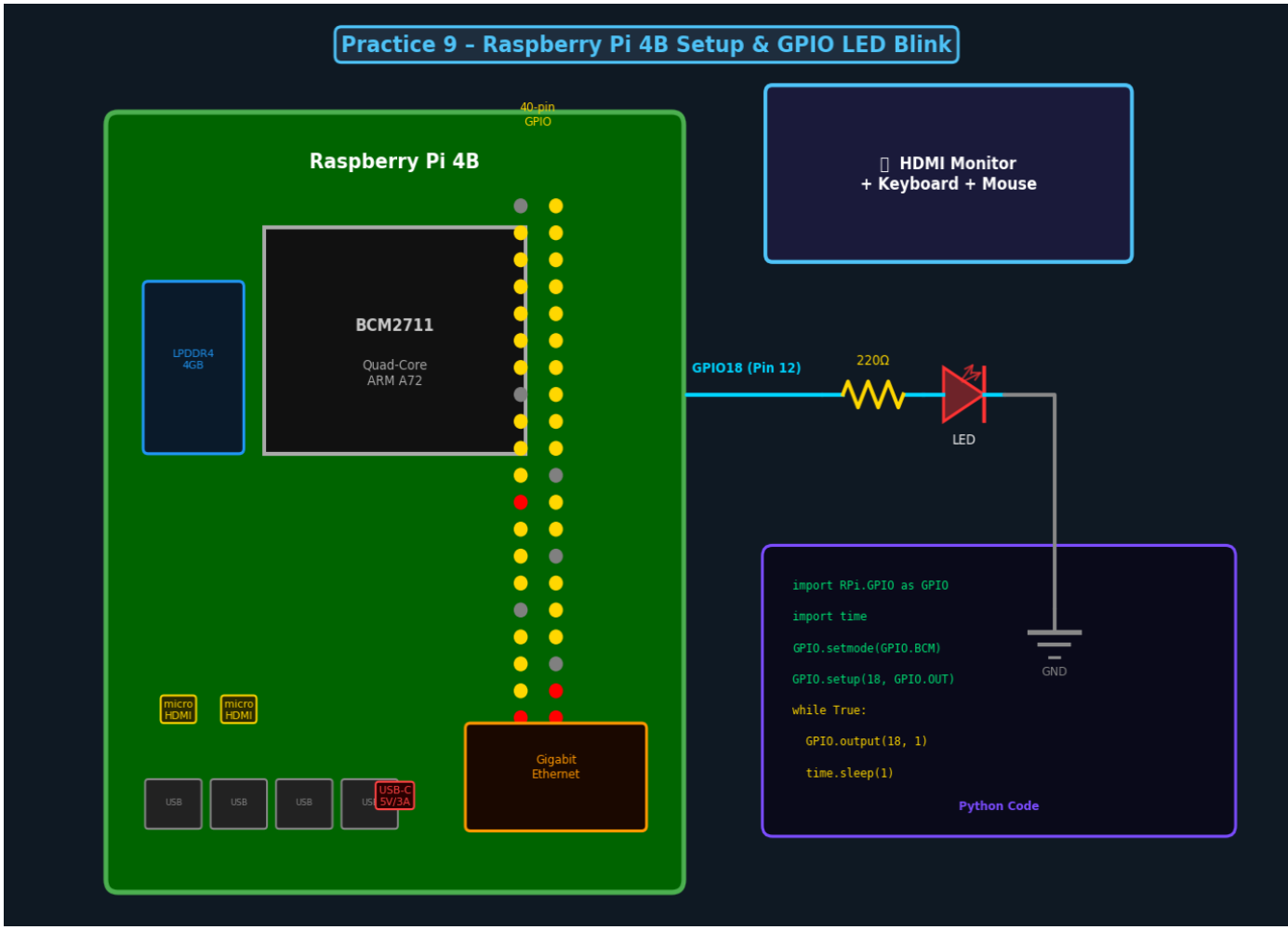


Fig 9.1 – Raspberry Pi 4B Board Layout & GPIO LED Circuit

Setup Steps

10. Flash Raspberry Pi OS using Raspberry Pi Imager → enable SSH + Wi-Fi.
11. Insert SD card, connect HDMI + keyboard + mouse, power on with 5V/3A USB-C.
12. Complete setup wizard: locale, password, Wi-Fi, software updates.
13. Enable GPIO/I2C/SPI via: sudo raspi-config → Interface Options.

Python LED Blink Code

```
# Practice 9: Raspberry Pi GPIO LED Blink
import RPi.GPIO as GPIO
import time

LED_PIN = 18  # BCM pin 18 = Physical pin 12
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)  # LED ON
        print("LED ON"); time.sleep(1)
        GPIO.output(LED_PIN, GPIO.LOW)   # LED OFF
        print("LED OFF"); time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Practice 10

Create a Webserver using NodeMCU ESP8266

Aim: To create an HTTP web server on NodeMCU that serves a control page to any browser on the same Wi-Fi network.

Duration: 3 Hours

Theory

NodeMCU's built-in TCP/IP stack allows it to operate as an HTTP server using the ESP8266WebServer library. It listens on port 80 and handles GET requests from browser clients. When a user clicks a button on the served HTML page, the browser sends a request (e.g., GET /led/on), the server processes it, toggles the LED, and redirects back to the main page.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	LED + 220Ω resistor	1
3	Breadboard & Jumper Wires	1 set
4	PC/Phone with Browser (same Wi-Fi)	1

System Diagram

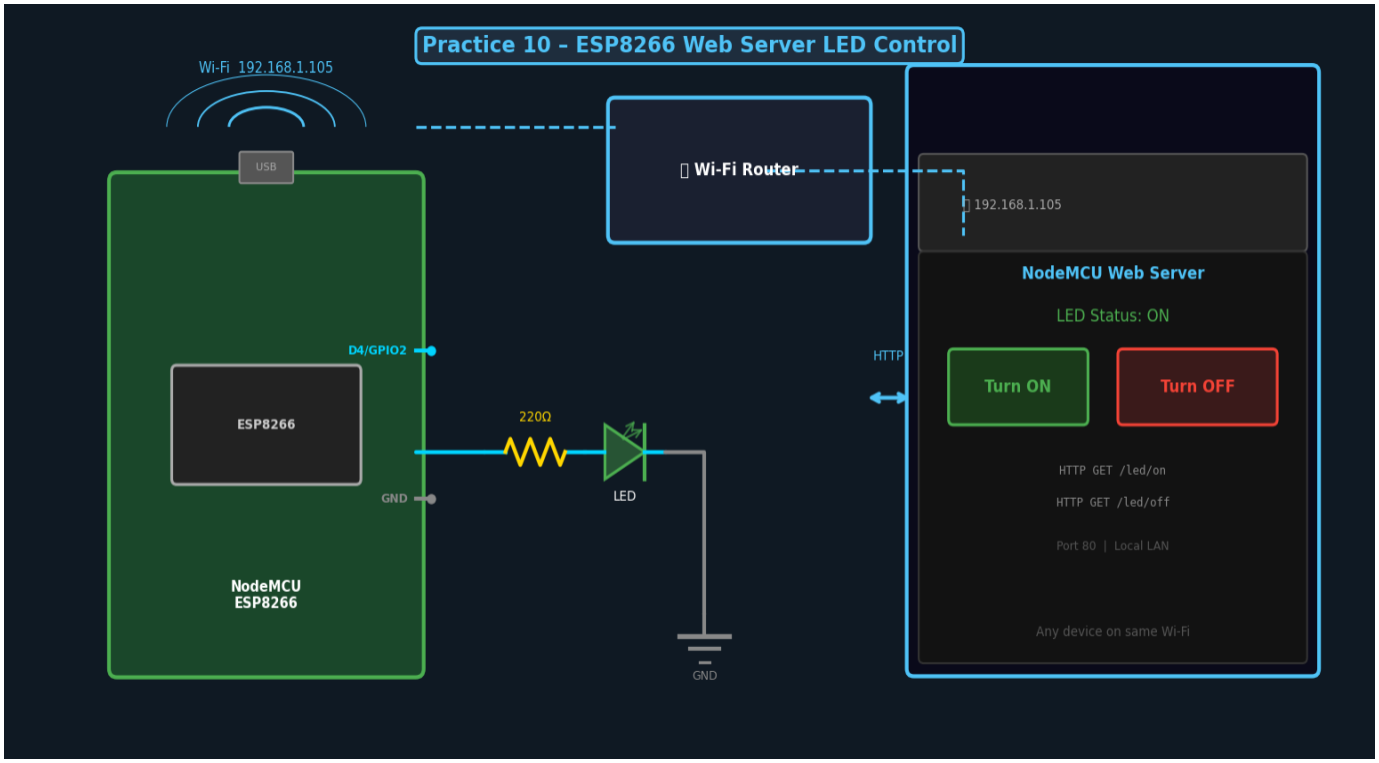


Fig 10.1 – ESP8266 Web Server – Browser LED Control

Program Code

```
// Practice 10: ESP8266 Web Server
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
const char* ssid = "Your_SSID";
```

```
const char* pass = "Your_Pass";
#define LED_PIN D4
bool ledOn = false;
ESP8266WebServer server(80);

String buildPage() {
  return "<html><body><h2>ESP8266 Server</h2><p>LED: " +
String(ledOn?"ON":"OFF")
    + "</p><a href='/led/on'><button>ON</button></a>"
    + "<a href='/led/off'><button>OFF</button></a></body></html>";
}
void handleRoot()   { server.send(200, "text/html", buildPage()); }
void handleLedOn()  { ledOn=true;  digitalWrite(LED_PIN,LOW);
server.sendHeader("Location","/"); server.send(302); }
void handleLedOff() { ledOn=false; digitalWrite(LED_PIN,HIGH);
server.sendHeader("Location","/"); server.send(302); }

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT); digitalWrite(LED_PIN, HIGH);
  WiFi.begin(ssid, pass);
  while (WiFi.status()!=WL_CONNECTED) delay(500);
  Serial.println("IP: " + WiFi.localIP().toString());
  server.on("/",        handleRoot);
  server.on("/led/on", handleLedOn);
  server.on("/led/off", handleLedOff);
  server.begin();
}
void loop() { server.handleClient(); }
```

Practice 11

Home Automation – Level 0 (Local Web Control)

Aim: To build a basic home automation system controlling 4 appliances via a local web dashboard.
Duration: 3 Hours

Theory

Level 0 Home Automation replaces physical switches with electronically-controlled relays managed through a local web interface. The NodeMCU creates a Wi-Fi hotspot or joins the home network, then serves a mobile-friendly HTML page showing appliance status buttons. Pressing a button sends a GET request that toggles the corresponding relay. No internet or cloud is required, making it fast and reliable.

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	4-Channel Relay Module (5V)	1
3	AC Appliances or LED indicators	4
4	5V Power Supply	1
5	Jumper Wires	1 set

System Diagram

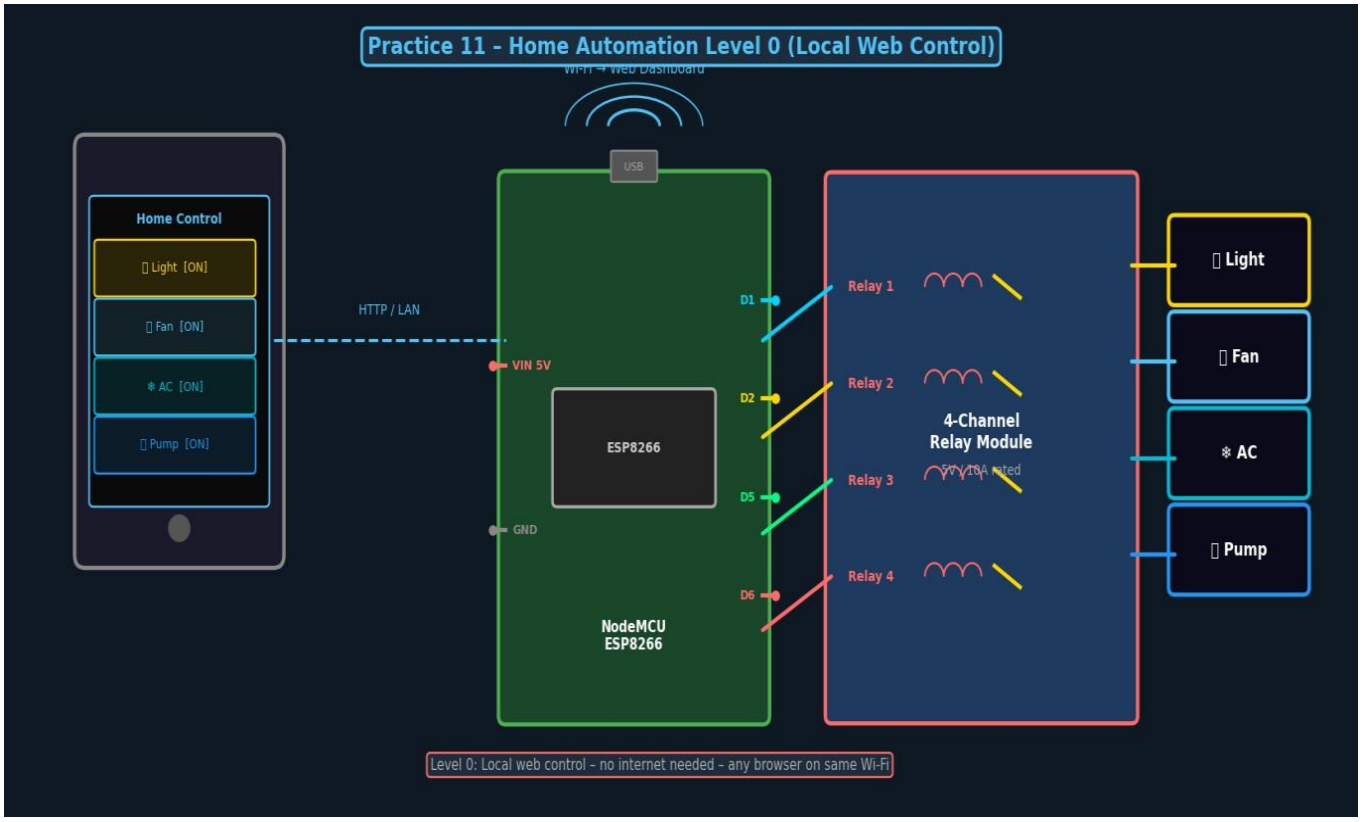


Fig 11.1 – Home Automation Level 0 – 4-Relay Web Dashboard

Program Code

```
// Practice 11: Home Automation Level 0
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
const char* ssid="SSID"; const char* pass="PASS";
int relayPins[]={D1,D2,D5,D6};
String apps[]={"Light 1","Light 2","Fan","AC"};
bool states[]={false,false,false,false};
ESP8266WebServer server(80);

void handleToggle() {
  int d = server.arg("dev").toInt();
  states[d] = !states[d];
  digitalWrite(relayPins[d], states[d]?LOW:HIGH);
  server.sendHeader("Location","/"); server.send(302);
}
void handleRoot() {
  String html = "<html><body><h2>Home Control</h2>";
  for(int i=0;i<4;i++)
    html+="<p>"+apps[i]+": "+String(states[i]?"ON":"OFF")
      +" <a href='/toggle?dev="+i+"'><button>Toggle</button></a></p>";
  html+="</body></html>";
  server.send(200,"text/html",html);
}
void setup() {
  for(int
i=0;i<4;i++){pinMode(relayPins[i],OUTPUT);digitalWrite(relayPins[i],HIGH);}
  WiFi.begin(ssid,pass);
  while(WiFi.status()!=WL_CONNECTED)delay(500);
  server.on("/",handleRoot); server.on("/toggle",handleToggle);
  server.begin();
}
void loop(){server.handleClient();}
```

Practice 12

Home Automation – Level 4 (Cloud + MQTT + Automation)

Aim: To build an advanced home automation system with MQTT cloud integration, sensor-triggered automation, and voice assistant compatibility.

Duration: 4 Hours

Theory

Level 4 Home Automation integrates: cloud connectivity (MQTT/Blynk), sensor-triggered rules (motion → light ON, temperature > 30°C → fan ON), time-based scheduling, energy monitoring, and voice assistant (Google/Alexa) integration via IFTTT. The MQTT protocol allows real-time pub/sub messaging between the NodeMCU and any cloud client.

System Architecture

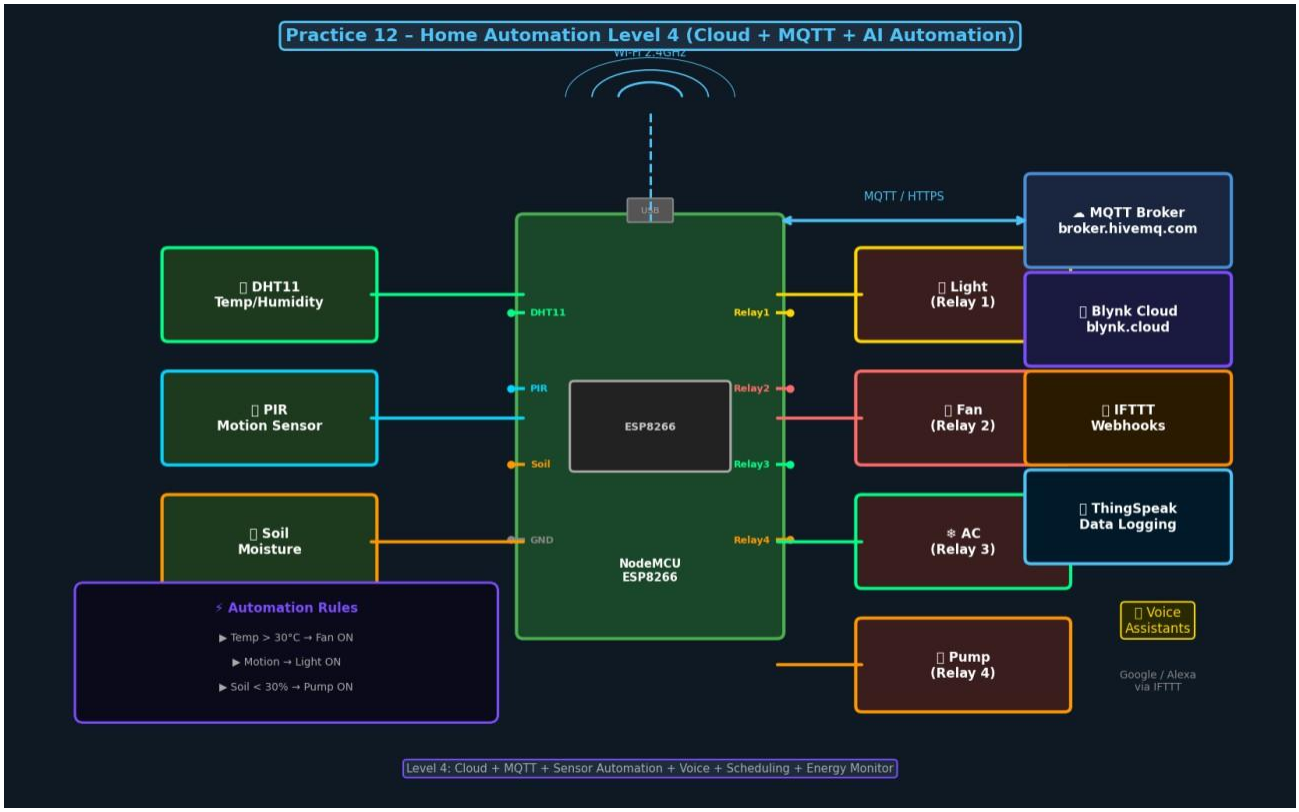


Fig 12.1 – Home Automation Level 4 Architecture: Cloud + MQTT + AI Automation

Program Code – MQTT + Sensor Automation

```
// Practice 12: Home Automation Level 4
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
const char* ssid="SSID"; const char* pass="PASS";
const char* broker="broker.hivemq.com";
#define DHT_PIN D5
#define PIR_PIN D7
#define RELAY_LIGHT D1
#define RELAY_FAN   D2
DHT dht(DHT_PIN, DHT11);
WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);
```

```
void mqttCallback(char* topic, byte* payload, unsigned int len) {
  String msg=""; for(int i=0;i<len;i++) msg+=(char)payload[i];
  if(String(topic)=="home/light") digitalWrite(RELAY_LIGHT,
msg=="ON"?LOW:HIGH);
  if(String(topic)=="home/fan")    digitalWrite(RELAY_FAN,
msg=="ON"?LOW:HIGH);
}

void setup() {
  pinMode(RELAY_LIGHT,OUTPUT); digitalWrite(RELAY_LIGHT,HIGH);
  pinMode(RELAY_FAN,  OUTPUT);  digitalWrite(RELAY_FAN,  HIGH);
  pinMode(PIR_PIN, INPUT);
  dht.begin();
  WiFi.begin(ssid,pass);
  while(WiFi.status()!=WL_CONNECTED) delay(500);
  mqtt.setServer(broker,1883); mqtt.setCallback(mqttCallback);
}

void loop() {
  if(!mqtt.connected()){mqtt.connect("HomeAuto");mqtt.subscribe("home/#");}
  mqtt.loop();
  if(digitalRead(PIR_PIN)==HIGH) digitalWrite(RELAY_LIGHT,LOW); // Motion
  float t=dht.readTemperature();
  if(!isnan(t)&&t>30.0) digitalWrite(RELAY_FAN,LOW);  // Hot -> Fan ON
  delay(2000);
}
```

Practice 13

Smart Irrigation System

Aim: To design an automatic irrigation system using NodeMCU ESP8266, soil moisture sensor, and a relay-controlled water pump with cloud monitoring.

Duration: 4 Hours

Theory

Traditional irrigation wastes up to 50% of water due to over-watering and fixed schedules. A smart irrigation system uses real-time soil moisture data to activate the pump only when needed. A float sensor prevents the pump from running dry. Environmental data (temperature, humidity) and moisture readings are uploaded to an MQTT cloud broker for monitoring and logging.

System Diagram

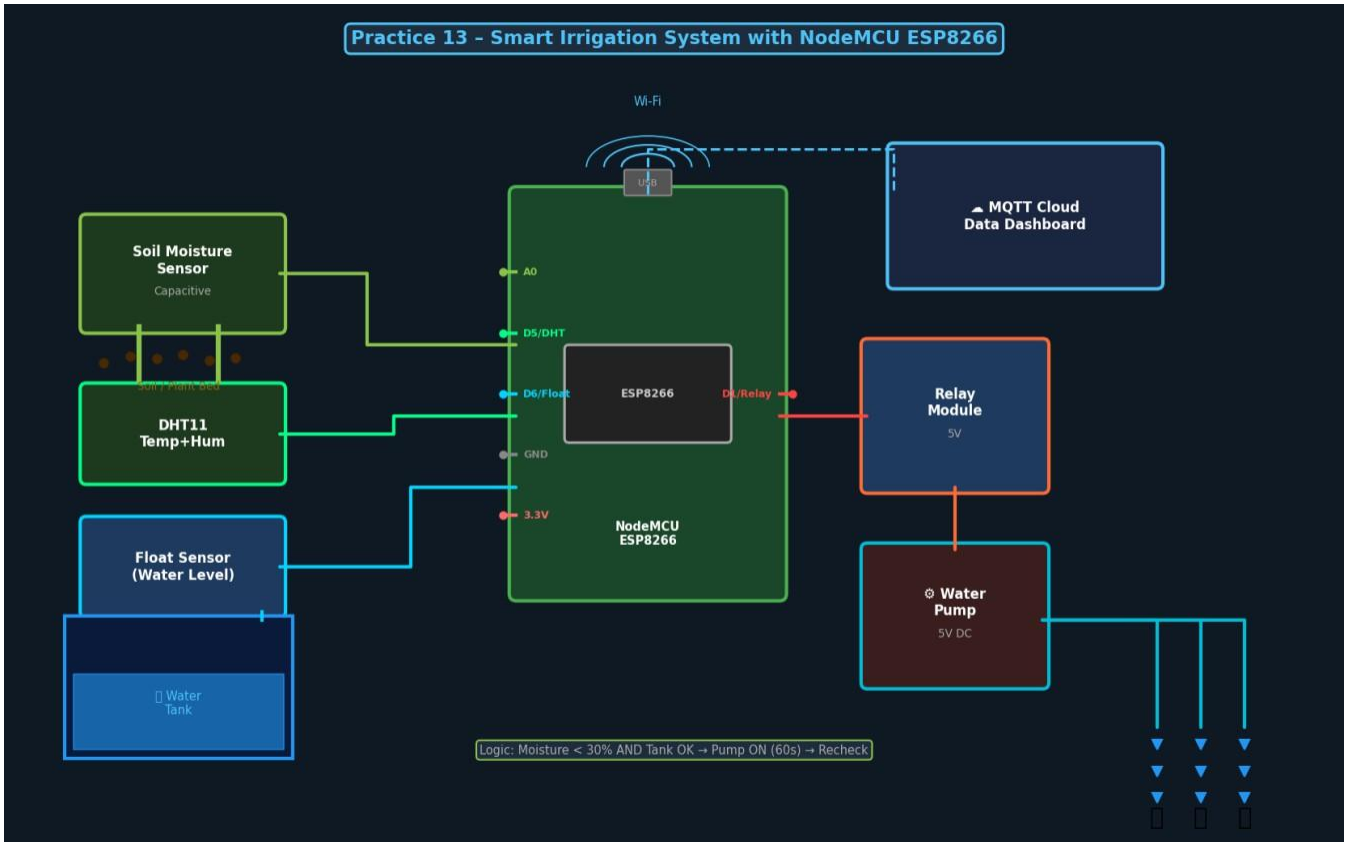


Fig 13.1 – Smart Irrigation System Architecture

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	Soil Moisture Sensor (Capacitive)	1–2
3	DHT11 Sensor	1
4	5V Relay Module	1
5	5V Water Pump (submersible)	1
6	Float Sensor (water level)	1

7	Water Tank + Tubing	1 set

Program Code

```
// Practice 13: Smart Irrigation System
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#define SOIL_PIN  A0
#define DHT_PIN   D5
#define FLOAT_PIN D6
#define PUMP      D1
#define DRY_PCT   30
DHT dht(DHT_PIN,DHT11);
WiFiClient wc; PubSubClient mqtt(wc);

void setup() {
  pinMode(PUMP,OUTPUT); digitalWrite(PUMP,HIGH);
  pinMode(FLOAT_PIN,INPUT_PULLUP);
  dht.begin();
  WiFi.begin("SSID","PASS");
  while(WiFi.status()!=WL_CONNECTED) delay(500);
  mqtt.setServer("broker.hivemq.com",1883);
  mqtt.connect("SmartIrrigation");
}

void loop() {
  mqtt.loop();
  float moist = map(analogRead(SOIL_PIN),1023,0,0,100);
  float temp  = dht.readTemperature();
  bool  tankOK = (digitalRead(FLOAT_PIN)==HIGH);

  Serial.printf("Moisture:%.0f%% Temp:%.1fC Tank:%s\n",
    moist, temp, tankOK?"OK":"EMPTY");

  if (moist < DRY_PCT && tankOK) {
    mqtt.publish("irrigation/event","Pump ON");
    digitalWrite(PUMP, LOW);   // Pump ON
    delay(60000);              // Run 60 seconds
    digitalWrite(PUMP, HIGH);  // Pump OFF
    mqtt.publish("irrigation/event","Pump OFF");
  } else if (!tankOK) {
    mqtt.publish("irrigation/alert","Water tank empty!");
  }
  delay(30000);
}
```

Practice 14

Weather Reporting System

Aim: To build an IoT weather station that uploads temperature, humidity, and light data to ThingSpeak cloud for visualization.

Duration: 3 Hours

Theory

A weather reporting system collects environmental parameters and uploads them to a cloud analytics platform. ThingSpeak (by MathWorks) accepts data via HTTP GET requests with an API key. The NodeMCU reads sensors every 20 seconds and sends readings to ThingSpeak channel fields, which are displayed as time-series charts. This forms the foundation of environmental monitoring and precision agriculture.

System Diagram

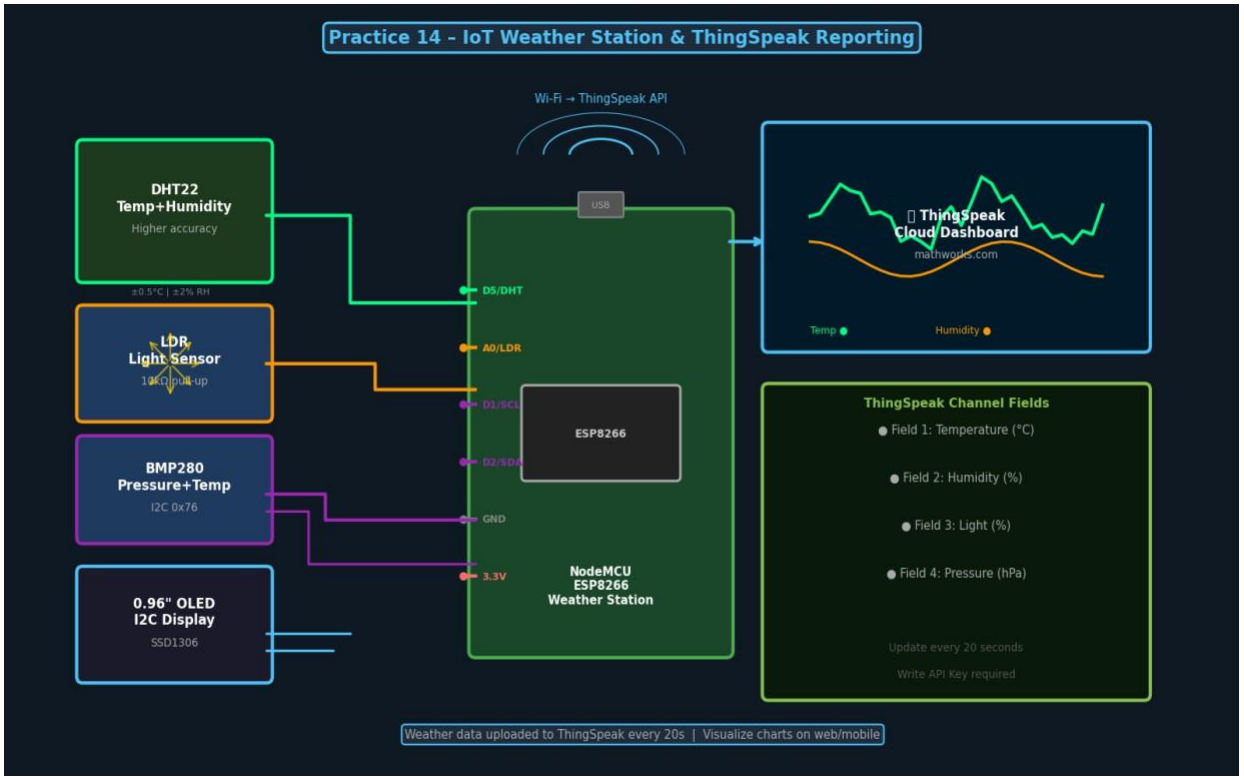


Fig 14.1 – IoT Weather Station with ThingSpeak Cloud Dashboard

Components Required

#	Component	Qty / Specification
1	NodeMCU ESP8266	1
2	DHT22 Sensor (higher accuracy)	1
3	LDR Module	1
4	BMP280 Pressure Sensor (optional)	1
5	0.96" OLED Display I2C (optional)	1

ThingSpeak Setup

14. Create account at thingspeak.com → New Channel → add fields: Temperature, Humidity, Light, Heat Index.
15. API Keys tab → copy Write API Key.
16. Paste key in code below.

Program Code

```
// Practice 14: Weather Station → ThingSpeak
#include <ESP8266WiFi.h>
#include  <DHT.h>
#define DHT_PIN D5
#define LDR_PIN A0
const char* ssid   = "Your_SSID";
const char* pass   = "Your_Pass";
const char* apiKey = "YOUR_THINGSPEAK_KEY";
DHT dht(DHT_PIN, DHT22);
WiFiClient client;

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(ssid,pass);
  while(WiFi.status()!=WL_CONNECTED){delay(500);Serial.print(".");}
  Serial.println("IP: "+WiFi.localIP().toString());
}

void loop() {
  float t=dht.readTemperature(), h=dht.readHumidity();
  int   light=map(analogRead(LDR_PIN),0,1023,0,100);
  if(client.connect("api.thingspeak.com",80)){
    String url="/update?api_key="+String(apiKey)
      +"&field1="+t+"&field2="+h+"&field3="+light;
    client.print("GET "+url+" HTTP/1.1\r\nHost: api.thingspeak.com\r\n");
    client.print("Connection: close\r\n\r\n");
    Serial.printf("Temp:%.1f H:%.0f Light:%d%% Uploaded!\n",t,h,light);
    client.stop();
  }
  delay(20000);  // ThingSpeak free: min 15s
}
```